
PILLAR

Smart Legislation — AI for Local Government

Comprehensive Technical Documentation
& Reference Guide

Version: 1.0.0

Date: June 2026

Repository: PILLAR Standalone

Target: Alibaba Cloud ECS with Docker

Platform for Intelligent Local Legislation and Administrative Reform.
AI-powered legislative support for Philippine Local Government Units.

Table of Contents

- 1. Executive Summary**
Design objectives, justification, and when to apply PILLAR
- 2. System Architecture**
High-level architecture, component diagram, data flows
- 3. Technology Stack**
Frontend, backend, AI/ML, and infrastructure
- 4. Knowledge Base Architecture**
BM25 + LightRAG hybrid search, citation validation
- 5. Authentication & Authorization**
Session-based auth, role-based access control
- 6. Admin Interface Guide**
Dashboard, KB management, deployments, ECS
- 7. Deployment Architecture**
Docker, nginx, multi-tenant LGU strategy
- 8. Database Schema**
Tables, indexes, SQLite configuration
- 9. API Reference**
All public, auth, and admin endpoints
- 10. Security Considerations**
Current posture and production hardening areas
- 11. Philippine Legal Domain Specificity**
LGU classification, bilingual processing
- 12. Roadmap & Future Enhancements**
Immediate, medium-term, and long-term priorities
- A. File Structure**
Complete project directory listing
- B. Default Configuration Values**
All tuning parameters and defaults

PILLAR: Smart Legislation — AI for Local Government

Comprehensive Technical Documentation & Reference Guide

Version: 1.0.0

Date: June 2026

Repository: PILLAR Standalone (decoupled from eSANGGUNI monorepo)

Target Deployment: Alibaba Cloud ECS with Docker

1. Executive Summary

What is PILLAR?

PILLAR (Platform for Intelligent Local Legislation and Administrative Reform) is an AI-powered web application designed to strengthen the legislative capacity of Local Government Units (LGUs) in the Philippines. It provides three specialized AI assistants that help Sangguniang Bayan members, LGU staff, and citizens navigate the complex landscape of local governance law.

PILLAR was originally built as a module within the eSANGGUNI monorepo, a broader digital governance platform. It has since been decoupled into a fully standalone application with its own AI backend, knowledge base, authentication system, and multi-tenant deployment infrastructure — requiring no external dependencies beyond an LLM API key.

Design Objectives

Objective 1: Standalone Independence. The primary design goal was complete operational independence from the eSANGGUNI backend. Every AI inference call, knowledge retrieval operation, and data persistence layer runs within the PILLAR container itself. This eliminates single points of failure and enables independent scaling, deployment, and versioning per LGU.

Objective 2: LightRAG-Primary Hybrid Search. Legal research demands both semantic understanding (what provisions relate to this concept?) and exact precision (what does Section 447(a)(1) say?). The retrieval architecture was redesigned to use LightRAG knowledge graph traversal as the primary search method, fused with BM25 full-text search for keyword precision. The two systems run in parallel, results are deduplicated and cross-validated, and the LLM receives a merged context that combines the strengths of both approaches.

Objective 3: Dynamic Knowledge Base. Philippine local governance law is not static. New ordinances are passed, DILG issues new opinions, and the Supreme Court renders new decisions continuously. The admin interface enables administrators to upload, index, and manage legal documents at runtime — enriching both the LightRAG graph and BM25 search index without rebuilding the application.

Objective 4: Multi-Tenant LGU Deployment. Each LGU deserves its own instance of PILLAR with its own ordinances, its own branding, and its own data isolation. The deployment architecture supports one dedicated Docker container per LGU, with multiple containers running on a single ECS instance, and one ECS instance per province. A central admin panel manages the entire fleet.

Objective 5: Defense-in-Depth Against Hallucination. AI-generated legal advice carries significant risk if citations are fabricated. PILLAR implements five layers of hallucination prevention: hardcoded ground-truth facts in prompts, hybrid retrieval grounding, citation allow-lists injected into prompts, post-generation regex-based citation validation, and structured JSON output for programmatic verification.

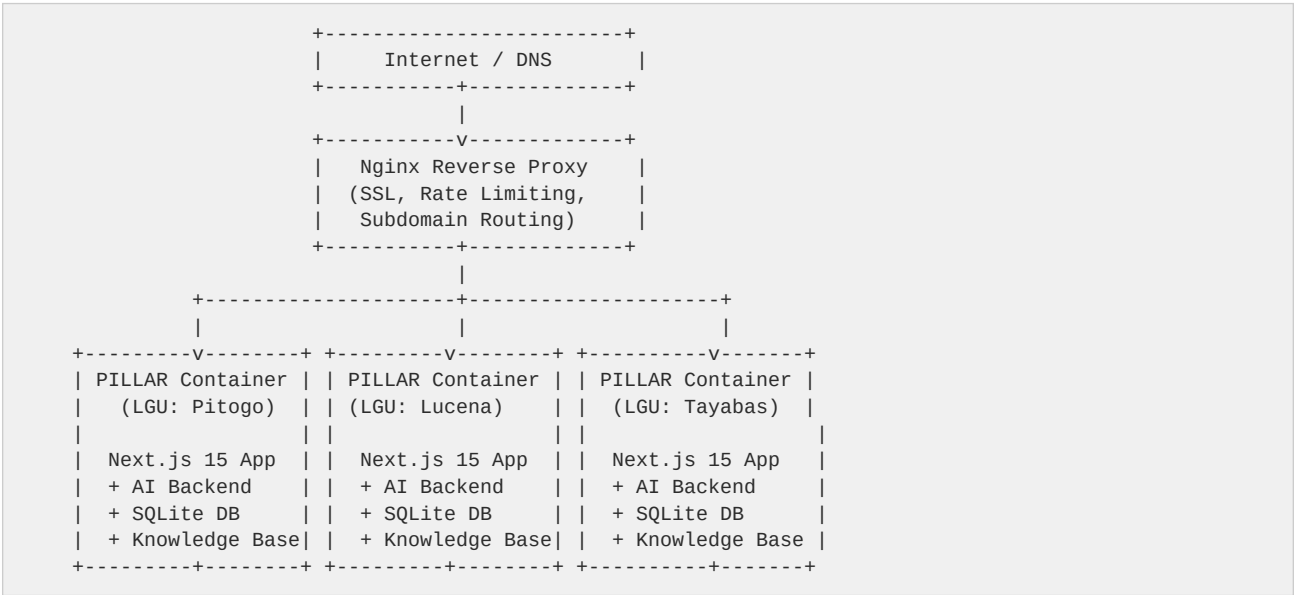
When to Apply PILLAR

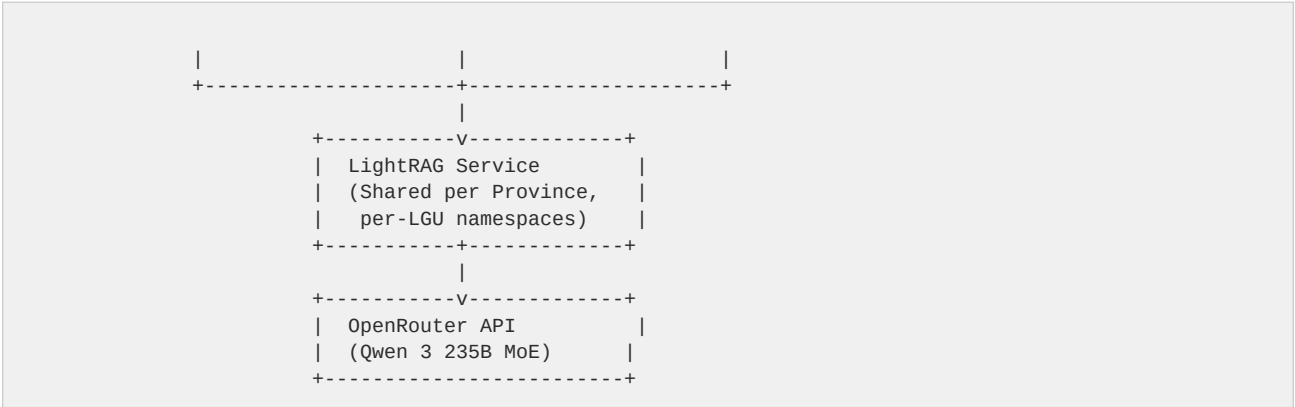
PILLAR is most effective in these scenarios:

- **Sangguniang Bayan Workshops** — Training sessions where SB members learn to draft ordinances, conduct legal research, and understand their legislative powers under R.A. 7160.
- **Day-to-Day Legislative Support** — Ongoing access to AI-assisted legal research and ordinance drafting for sitting legislators and SB secretaries.
- **Multi-LGU Rollout Programs** — Provincial or regional initiatives to deploy standardized AI legislative tools across all municipalities and cities.
- **Citizen-Facing LGU Information** — The YALA chatbot provides residents with accurate information about municipal services, requirements, fees, and processes.

2. System Architecture

High-Level Architecture





Component Architecture

Each PILLAR container is a self-contained Next.js 15 application with these internal components:

Public-Facing Modules:

- **ELLA** (Electronic Legal Legislative Assistant) — AI legal research assistant for legislators. Handles queries about R.A. 7160, municipal ordinances, IRR provisions, DILG opinions, and SC jurisprudence. Returns streaming responses with verified citations.
- **OBRA** (Ordinance Builder and Review Assistant) — AI ordinance drafting and compliance review tool. Generates draft ordinances from templates (tax, regulatory, appropriation, general) and reviews existing drafts for legal compliance, structural completeness, and citation accuracy.
- **YALA** (Your Assistant for Local Affairs) — Citizen-facing chatbot providing information about municipal services (business permits, barangay clearance, civil registry, property tax, etc.). Bilingual (English/Filipino). Tightly scoped to prevent out-of-domain responses.

Admin System:

- **Authentication** — Cookie-based session auth with two roles: Super Admin (system-wide access) and LGU Admin (scoped to assigned LGU/province).
- **Knowledge Base Manager** — Upload, view, filter, re-index, and delete legal documents. Supports ordinances, R.A. 7160 sections, IRR rules, DILG opinions, SC jurisprudence, policies, and LGU context files.
- **Deployment Manager** — Create and manage per-LGU Docker container deployments. Lifecycle actions: deploy, start, stop, restart, rebuild, destroy.
- **ECS Infrastructure Manager** — Register and monitor Alibaba Cloud ECS instances. Health checks, credential management, deployment counts.

AI Backend:

- **Hybrid RAG Pipeline** — Orchestrates LightRAG graph queries and BM25 full-text search in parallel, merges results with deduplication and cross-validation, and assembles context for LLM injection.
- **LLM Gateway** — OpenAI-compatible client pointed at OpenRouter. Supports streaming (SSE) for chat and non-streaming for structured tasks.
- **Citation Validator** — Post-generation regex extraction and index-based verification of legal citations. Returns verified/unverified partitions with confidence levels.

Data Flow: ELLA Query

A typical ELLA legal research query follows this path:

1. User submits a question via the ELLA chat interface.
2. The `/api/chat` endpoint receives the message with conversation history.
3. The hybrid RAG pipeline fires two parallel queries: LightRAG graph traversal (mode: "hybrid", top_k: 8) and BM25 full-text search (6 results with section/ordinance boosting).
4. Results are merged: LightRAG sources take priority, BM25 results that overlap are deduplicated, and only complementary BM25 results are appended. Cross-validated documents (found by both systems) receive a confidence boost.
5. The merged context (up to 4,000 characters) is injected into the ELLA system prompt.
6. The LLM generates a streaming response via OpenRouter (Qwen 3 235B MoE, temperature 0.3).
7. After streaming completes, the citation validator extracts all legal citations from the response using regex patterns and verifies each against the search index.
8. The final SSE chunk includes citation metadata with verification status and confidence levels.
9. The interaction is logged to SQLite (non-blocking) for workshop analytics.

Data Flow: OBRA Draft Generation

1. User selects a template type (tax, regulatory, appropriation, general) and provides details (title, subject matter, key provisions, target area).
2. BM25 search retrieves 8 relevant documents. LightRAG provides semantic context.
3. A citation allow-list is built from the RAG results, constraining the LLM to only cite provisions that were actually retrieved.
4. The LLM generates a complete ordinance draft (non-streaming, temperature 0.5, 4096 tokens).
5. Post-generation citation validation flags any unverified or potentially hallucinated citations.
6. The response includes the full draft text plus citation warnings for the user to review.

3. Technology Stack

Frontend

Technology	Version	Purpose
Next.js (App Router)	15.2.4	Full-stack React framework with server-side rendering and API routes
React	19.0.0	UI component library
Tailwind CSS	3.4	Utility-first CSS framework
Recharts	2.12.7	Charting library for admin analytics dashboard

Technology	Version	Purpose
Lucide React	0.468.0	Icon library
Base UI (Radix)	1.4.1	Headless accessible UI primitives (Dialog, Tabs, Sheet)

Backend

Technology	Version	Purpose
better-sqlite3	9.6.0	Synchronous SQLite3 driver with native bindings. WAL mode enabled for concurrent reads.
OpenAI SDK	6.35+	LLM API client, configured for OpenRouter compatibility layer
gray-matter	4.0.3	Front-matter parsing for document ingestion (ordinance files with YAML headers)
tsx	4.19+	TypeScript execution engine for running ingestion scripts

AI / Machine Learning

Component	Details
LLM Provider	OpenRouter (model-agnostic gateway)
Default Model	Qwen 3 235B MoE (qwen/qwen3-235b-a22b) — 22B active parameters, 128K context
Temperature	0.3 for chat (ELLA, YALA), 0.5 for drafting (OBRA), 0.1 for compliance review
BM25 Search	Custom implementation with bilingual stop words (English + Filipino), IDF scoring, section/ordinance number boosting (5x), topic tag boosting (1.5x)
LightRAG	External Docker service for knowledge graph traversal. Entity extraction, relation mapping, hybrid local+global queries. Optional — system degrades gracefully to BM25-only.
Citation Validation	Regex-based extraction of R.A. 7160 sections, municipal ordinances, and IRR rules. Index-based verification against the knowledge base.

Infrastructure

Component	Details
Container Runtime	Docker with multi-stage Alpine-based builds
Reverse Proxy	Nginx with SSL (Let's Encrypt), rate limiting, SSE-optimized proxy settings
Cloud Provider	Alibaba Cloud ECS (one instance per province)
Database	SQLite (single file per container, persisted via Docker volume)
SSL/TLS	Automated via Certbot with 12-hour renewal checks

Component	Details
Process Model	Single Node.js process per container (Next.js standalone server)

4. Knowledge Base Architecture

Document Corpus

The PILLAR knowledge base currently contains 2,634 legal documents across five categories:

Category	Count	Source	ID Pattern
R.A. 7160 Sections	100	Local Government Code of 1991	ra7160-sec-{number}
Municipal Ordinances	340	Pitogo, Quezon (1989--2025)	ord-{type}-{number}-S{year}
IRR Rules	39	Implementing Rules of R.A. 7160	irr-irr-rule-{roman}
DILG Legal Opinions	1,562	DILG Legal Service (2005--2025)	lo-lo-{number}-s{year}
SC Jurisprudence	161	Supreme Court decisions	sc-sc-gr-{number}

Each document is stored as an individual JSON file containing `{ id, full_text }` at `src/lib/data/documents/`. The total corpus is approximately 28 MB.

BM25 Search Index

The search index (`src/lib/data/search-index.json` , ~12 MB) is a pre-built BM25 inverted index containing:

- **Document records** with term frequency maps, metadata (doc_type, title, section_number, ordinance_number, series_year, rule_number, topics, book, chapter, snippet).
- **IDF vocabulary** computed using the standard BM25 formula: $\log((N - df + 0.5) / (df + 0.5) + 1)$.
- **Average document length** for length normalization (parameter B = 0.75).
- **Bilingual stop words** — 100+ English and 30+ Filipino/Tagalog stop words filtered during tokenization.

The index is built offline via `npm run ingest` and baked into the Docker image at build time.

LightRAG Knowledge Graph

LightRAG is an optional external service that provides graph-based semantic retrieval. When available, it performs entity extraction and relation mapping across the legal corpus, enabling queries like "what provisions relate to the power of the Sangguniang Bayan to approve appropriations?" to surface relevant sections even when the exact keywords don't match.

Deployment model: One LightRAG container per province (shared across all LGU PILLAR instances on the same ECS), with per-LGU namespaces to maintain data isolation. This is significantly more memory-efficient than dedicated LightRAG instances (~3 GB shared vs. ~120 GB for 40 dedicated instances).

Graceful degradation: If the LightRAG service is unavailable, the hybrid pipeline automatically falls back to BM25-only search. The response header is marked `[DEGRADED MODE]` to signal that graph-based retrieval was not available.

Hybrid Search Strategy: LightRAG-Primary with BM25 Fusion

The `getHybridContext()` function in `src/lib/ai/rag.ts` implements the core retrieval strategy:

Step 1 — Parallel Execution: Both LightRAG and BM25 queries fire simultaneously via `Promise.all`, avoiding sequential latency.

Step 2 — LightRAG as Primary: The LightRAG response (mode: "hybrid", combining local entity neighborhoods with global graph traversal) is treated as the primary context source.

Step 3 — BM25 as Complementary: BM25 results are filtered against LightRAG sources. Documents already covered by LightRAG (matched by document ID) are excluded. Only complementary BM25 results — those providing exact keyword matches that the graph may have missed — are appended.

Step 4 — Smart Merge with Budget: A 4,000-character context budget is allocated 70/30 between LightRAG and BM25 complementary results. Each result is formatted with headers, document type labels, and relevance indicators.

Step 5 — Cross-Validation Signal: Documents found by both systems are tracked as `crossValidatedCount` and reported in the context header as a confidence indicator (HIGH vs. MEDIUM).

Domain-Specific Search Boosting

The BM25 search applies domain-aware scoring adjustments that significantly improve retrieval quality for Philippine legal documents:

- **Section number match (5x boost):** Queries containing "Section 447" or "Sec. 447" receive a 5x score multiplier for documents matching that section.
- **Ordinance number match (5x boost):** Queries referencing specific ordinance numbers (with zero-padding normalization for variants like "MO-1" vs. "MO-001") receive a 5x multiplier.

- **Topic tag match (1.5x boost):** Documents whose indexed topic tags match query terms receive a 1.5x multiplier.

Citation Validation System

After the LLM generates a response, the citation validator (`src/lib/ai/citation-validator.ts`) performs post-hoc verification:

Extraction: Three regex patterns extract citations from the generated text:

- R.A. 7160 sections: `Section XXX` or `Sec. XXX` with optional sub-sections like `(a)(3)(iii)`.
- Municipal ordinances: `Municipal Ordinance No. XX, S. YYYY`, `MO No. XX, S. YYYY`, `AO No. XX, S. YYYY`.
- IRR rules: `IRR Rule XX` with Roman numeral conversion (I through XXXIX).

Verification: Each extracted citation is looked up in the BM25 search index:

- R.A. 7160 sections are matched by section number. Confidence is HIGH for base sections, MEDIUM when sub-sections are referenced (sub-section text cannot be verified from the index alone), LOW if not found.
- Ordinances are matched by ordinance number (with zero-padding normalization) AND series year.
- IRR rules are matched by converting Roman numerals to integers and looking up the rule number.

Output: The validation result partitions citations into verified and unverified sets, with per-citation confidence levels and document references.

Anti-Hallucination: Hardcoded Ground Truths

The system prompts for ELLA, OBRA Draft, and OBRA Review all include an "IMPORTANT FACTS" section that pins down commonly hallucinated details:

- The IRR of R.A. 7160 has exactly 39 rules.
- Pitogo has 349 ordinances spanning 1989--2025.
- Section 15 concerns corporate personality of LGUs (NOT penal provisions).
- Section 447 defines Sangguniang Bayan powers; Section 444 defines provincial powers (NOT municipal).
- "Section 44(a)(3)(i)" does not exist in R.A. 7160 — a common LLM hallucination.

These facts are repeated verbatim across prompts to ensure the LLM internalizes them regardless of context window position.

5. Authentication & Authorization

Session-Based Authentication

PILLAR uses server-side cookie-based sessions rather than JWTs, providing immediate revocation capability and avoiding token leakage risks.

Login flow:

- 1. User submits credentials to `POST /api/auth/login` .
- 2. Password is verified using `crypto.scryptSync` with a 16-byte random salt (64-byte derived key).
- 3. A 32-byte random hex session token is generated and stored in the `admin_sessions` table with a 24-hour expiration.
- 4. The token is set as an `httpOnly` , `secure` (in production), `sameSite=lax` cookie named `pillar_session` .

Session validation: Every authenticated API request reads the `pillar_session` cookie, looks up the session in the database (JOIN with `admin_users`), and verifies expiration and active status. Deactivated users are immediately locked out even with a valid session.

Role-Based Access Control

Two roles are supported:

Role	Scope	Access
Super Admin	System-wide	All admin features: KB management, deployment management, ECS management, credentials, analytics
LGU Admin	Assigned LGU/province	KB management (filtered by assigned LGU), analytics. No access to deployments or ECS.

The `withAuth()` middleware wrapper supports a `{ requireSuperAdmin: true }` option that returns 403 Forbidden for non-super-admin users. This is applied to deployment, ECS, and credential management routes.

Default Credentials

On first startup, if no admin users exist, the system seeds a default Super Admin account:

- Username: `admin`
- Password: `pillar-admin-2026`
- Role: `super_admin`

This should be changed immediately after first login in a production deployment.

6. Admin Interface Guide

Login

Navigate to `/admin/login` . Enter your username and password. The session cookie is set on successful authentication and you are redirected to the admin dashboard.

Dashboard (`/admin`)

The admin dashboard provides quick-access navigation cards and workshop analytics:

- **Quick Access Cards:** Links to Knowledge Base (all admins), Deployments (super admin only), and ECS Management (super admin only).
- **Workshop Analytics:** Displays session statistics, top topics by module (ELLA, OBRA, YALA), a session timeline chart, OBRA insights (average compliance scores, most common risks), and a table of the most active participants.
- **Auto-refresh:** Statistics refresh every 30 seconds during active workshops.

Knowledge Base Management (`/admin/kb`)

The KB management page allows administrators to view, filter, and manage the legal document corpus:

Statistics Bar: Shows total documents, indexed count, pending count, and LightRAG entity/relation counts. A LightRAG health indicator shows whether the graph service is connected.

Filtering: Documents can be filtered by search text (client-side title/ordinance/section match), document type (ordinance, ra7160, irr, dilg_opinion, jurisprudence, policy, context), indexing status (pending, indexed, error), and LGU ID.

Document Table: Paginated list (20 per page) showing document title, type, status badge, LGU ID, and last updated timestamp. Each row has View and Delete actions.

View Modal: Displays full document metadata including content preview (first 2,000 characters), topics, section/ordinance/rule numbers, and any error messages from failed indexing attempts.

Re-index: Triggers LightRAG ingestion for all pending documents within a specified LGU. The re-index process creates a `kb_index_runs` record, iterates documents calling the LightRAG ingest API, and updates each document's status to `indexed` or `error` .

Document Upload (`/admin/kb/upload`)

The upload form accepts new legal documents for inclusion in the knowledge base:

Required Fields:

- **LGU ID** — Identifier for the LGU this document belongs to (e.g., `pitogo-quezon`).
- **Document Type** — One of: ordinance, ra7160, irr, dilg_opinion, jurisprudence, policy, context.
- **Title** — Human-readable document title.

Optional Metadata:

- Section Number (for R.A. 7160 sections)
- Ordinance Number and Series Year (for ordinances)
- Ordinance Type (e.g., tax, regulatory)
- Rule Number (for IRR rules)
- Topics (comma-separated tags)

File Upload: Supports drag-and-drop and click-to-browse. Accepted formats: `.json`, `.txt`, `.md`, `.pdf`. The file is saved to `src/lib/data/documents/` and the content is extracted (JSON parsing for `.json`, raw text for `.txt` / `.md`). Documents are created with status `pending` and require a re-index operation to be ingested into LightRAG.

Deployment Manager (`/admin/deployments`)

The deployment manager provides a fleet overview of all LGU container deployments:

Statistics Cards: Total deployments, Running, Stopped, Error, and Building counts.

Filtering: By province and deployment status.

Deployment Cards: Grid layout showing each LGU's name, type (municipality/city/HUC), income class (1st--6th), province, assigned ECS instance, container name and port, custom domain, last deployed timestamp, and any error messages.

Actions: Start, Stop, Restart, Rebuild, and Delete. Each action generates the corresponding Docker commands and returns the SSH connection details for the target ECS. Note: as of v1.0, commands are generated but not automatically executed via SSH — this is planned for a future release.

New Deployment (`/admin/deployments/new`): Form to register a new LGU deployment. Fields include LGU name (auto-generates LGU ID and container name), type, class, province, target ECS instance, container configuration, domain, and OpenRouter API key.

ECS Infrastructure Manager (`/admin/ecs`)

The ECS manager provides visibility into the Alibaba Cloud infrastructure:

Instance Cards: Each ECS instance shows its name, IP address, province/region, status badge, deployment counts (running/total), SSH user, and last health check time.

Health Check: Triggers a health check API call. Currently returns container counts from the database; CPU, memory, disk, and Docker version metrics are planned for SSH-based implementation.

Delete: Only allowed when an ECS instance has zero deployments (prevents orphaned containers).

New ECS Instance (`/admin/ecs/new`): Form to register a new ECS server with name, public IP, province, region, SSH configuration (port, user, key path), and notes.

7. Deployment Architecture

Container Model: One Container per LGU

Each LGU gets a dedicated Docker container running its own PILLAR instance with its own:

- Application codebase (Next.js standalone bundle)
- SQLite database (workshop sessions, participant logs, admin users)
- Knowledge base (ordinances, DILG opinions, local context files specific to that LGU)
- BM25 search index (pre-built with the LGU's documents)
- Workshop analytics and interaction logs

This isolation ensures data sovereignty (each LGU's data is physically separate), enables independent scaling and updates, and prevents cross-LGU interference.

ECS Allocation: One ECS per Province

Each Alibaba Cloud ECS instance hosts multiple LGU containers for all municipalities and cities within a single province. For a province like Quezon with 40+ LGUs, a single ECS with 64 GB RAM and 16 vCPUs can comfortably handle the workload since each PILLAR container is lightweight (Next.js standalone with SQLite, ~100--200 MB RAM per container).

LightRAG Sharing: One Instance per Province

Rather than deploying a dedicated LightRAG container per LGU (which would require ~3 GB each, totaling 120+ GB for a large province), a single LightRAG instance per ECS serves all LGU containers on that host with per-LGU namespaces for data isolation. This reduces the LightRAG memory footprint from ~120 GB to ~3 GB per province.

Docker Build

The Dockerfile uses a multi-stage Alpine-based build:

Stage 1 (Builder): Installs native compilation tools for `better-sqlite3`, runs `npm ci`, and produces the Next.js standalone output via `npm run build`. Build arguments accept public URLs for ELLA, OBRA, YALA, and PILLAR (defaulting to relative paths).

Stage 2 (Runner): Minimal image with only the standalone output, a non-root `nextjs` user (UID 1001), and a `/data` directory for SQLite persistence. The SQLite database lives at `/data/pillar.db` and is persisted via a Docker named volume.

Nginx Configuration

Nginx serves as the reverse proxy with four subdomain-based server blocks:

Subdomain	Purpose
<code>pillar.bayanaihan.net</code>	Landing page and main navigation
<code>ella.pillar.bayanaihan.net</code>	ELLA legal research assistant
<code>obra.pillar.bayanaihan.net</code>	OBRA ordinance drafting and review
<code>yala.pillar.bayanaihan.net</code>	YALA citizen information chatbot

All four subdomains proxy to the same `pillar:3000` upstream. Key proxy settings include:

- **SSE optimization:** `proxy_buffering off` and `proxy_read_timeout 300s` for long-running AI chat streams.
- **Rate limiting:** 10 req/s for general API endpoints, 5 req/min for login (per IP).
- **SSL:** Automated Let's Encrypt certificates with 12-hour renewal checks via Certbot.
- **WebSocket support:** Upgrade headers configured for potential future WebSocket use.

Environment Variables

Variable	Required	Purpose
<code>OPENROUTER_API_KEY</code>	Yes	API key for OpenRouter LLM gateway. Without this, all AI features return 503.
<code>LIGHTRAG_SERVICE_URL</code>	No	LightRAG service URL. If unset, the system falls back to BM25-only search.
<code>DB_PATH</code>	No	SQLite database path. Default: <code>/data/pillar.db</code> (matches Docker volume mount).
<code>LLM_MODEL</code>	No	Override the default LLM model. Default: <code>qwen/qwen3-235b-a22b</code> .

8. Database Schema

Tables

workshop_sessions — Top-level workshop session records.

Columns: `id` (UUID PK), `title`, `started_at`, `ended_at`.

participant_sessions — Individual participant registrations within a workshop.

Columns: `id` (UUID PK), `workshop_session_id` (FK -> `workshop_sessions`), `name`, `ip_address`, `module`, `created_at`, `last_active_at`.

interaction_logs — All chat messages, draft generations, and review interactions.

Columns: `id` (UUID PK), `participant_session_id` (FK), `module`, `interaction_type`, `user_message`, `assistant_response`, `metadata` (JSON: topic tags, compliance scores, draft lengths), `created_at`.

admin_users — Administrator accounts.

Columns: `id` (UUID PK), `username` (unique), `password_hash` (script salt:hash format), `display_name`, `role` (`super_admin` | `lgu_admin`), `lgu_id`, `province_id`, `is_active`, `created_at`, `last_login_at`.

admin_sessions — Active login sessions.

Columns: `id` (TEXT PK, 32-byte hex token), `user_id` (FK -> `admin_users`, CASCADE), `expires_at`, `created_at`.

kb_documents — Knowledge base document registry.

Columns: `id` (UUID PK), `lgu_id`, `doc_type` (7 possible values), `title`, `content` (TEXT), `file_path`, `metadata` (JSON), `section_number`, `ordinance_number`, `series_year`, `ordinance_type`, `rule_number`, `topics` (JSON array), `status` (`pending` | `indexed` | `error`), `error_message`, `created_by`, `created_at`, `updated_at`.

kb_index_runs — LightRAG indexing batch records.

Columns: `id` (UUID PK), `lgu_id`, `status` (`running` | `completed` | `error`), `documents_indexed`, `error_message`, `started_at`, `completed_at`.

ecs_instances — Alibaba Cloud ECS server inventory.

Columns: `id` (UUID PK), `name`, `province`, `region`, `public_ip`, `ssh_port`, `ssh_user`, `ssh_key_path`, `ssh_key_encrypted`, `api_token`, `status` (`online` | `offline` | `error` | `unknown`), `last_health_check`, `notes`, `created_at`, `updated_at`.

lgu_deployments — Per-LGU container deployment records.

Columns: `id` (UUID PK), `lgu_id`, `lgu_name`, `lgu_type` (4 LGU classifications), `lgu_class` (1st--6th income class), `province`, `ecs_instance_id` (FK -> `ecs_instances`), `container_name`, `container_port`, `domain`, `openrouter_api_key`, `image_tag`, `lightrag_service_url`, `env_vars` (JSON), `status` (`running` | `stopped` | `error` | `building` | `unknown`), `health_status`, `error_message`, `last_deployed_at`, `created_at`, `updated_at`.

Indexes (15 total)

Strategic indexes on the most frequently queried columns: `workshop_session_id`, `module`, `created_at`, `interaction_type` on interaction logs; `workshop_session_id` on participant sessions; `user_id` and `expires_at` on admin sessions; `lgu_id`, `doc_type`, and `status` on KB documents; `lgu_id` on index runs; and `ecs_instance_id`, `lgu_id`, `status`, and `province` on deployments.

SQLite Configuration

- **WAL mode** enabled for concurrent read performance.
 - **Foreign keys** enabled via `PRAGMA foreign_keys = ON`.
 - **Synchronous mode** set to `NORMAL` for balanced durability and write performance.
 - **Data directory** auto-created at `/data` on first startup.
-

9. API Reference

Public Endpoints

POST /api/chat — AI chat for ELLA and YALA modules.

Accepts: `module` (ella|yala), `message`, `history`, `participantName`, `sessionId`.

Returns: Server-Sent Events stream with text chunks and citation metadata.

POST /api/obra/draft — Generate ordinance draft.

Accepts: `template` (tax|regulatory|appropriation|general), `details` (title, subjectMatter, keyProvisions, targetArea), `participantName`, `sessionId`.

Returns: `{ draft, citationWarnings }`.

POST /api/obra/review — Review ordinance compliance.

Accepts: `draft` (full text), `participantName`, `sessionId`.

Returns: `ReviewResult` with overallScore, risks, structuralIssues, missingSections, citationWarnings.

GET /api/health — Service health check.

Returns: `{ status, service, modules, lightrag, timestamp }`.

GET /api/documents/[id] — Full document text retrieval.

Accepts: Document ID (validated against ra7160-sec-, ord-, irr-* patterns).

Returns: `{ id, full_text }`.

Authentication Endpoints

POST /api/auth/login — Authenticate and create session.

Accepts: `{ username, password }`.

Returns: User object + sets `pillar_session` cookie.

POST /api/auth/logout — Destroy session and clear cookie.

GET /api/auth/me — Current user from session cookie.

Returns: `{ user, expires_at }` or 401.

Admin Knowledge Base Endpoints

GET /api/admin/kb/documents — List documents with filters and pagination.

Query params: `lgu_id`, `doc_type`, `status`, `page`, `limit`.

POST /api/admin/kb/documents — Create document record.

Body: `{ lgu_id, doc_type, title, content?, metadata?, ... }`.

GET /api/admin/kb/documents/[id] — Single document detail.

PUT /api/admin/kb/documents/[id] — Update document metadata or content.

DELETE /api/admin/kb/documents/[id] — Delete document (also removes from LightRAG and disk).

POST /api/admin/kb/upload — Multipart file upload.

Accepts: `lgu_id`, `doc_type`, `title`, `file` (.json/.txt/.md/.pdf), optional metadata.

POST /api/admin/kb/reindex — Trigger LightRAG ingestion for pending documents.

Body: `{ lgu_id }`.

GET /api/admin/kb/stats — Aggregate KB statistics and LightRAG stats.

Admin Deployment Endpoints

GET /api/admin/deployments — List deployments with ECS join.

Query params: `ecs_id`, `status`, `province`, `page`, `limit`.

POST /api/admin/deployments — Create deployment (super admin only).

GET /api/admin/deployments/[id] — Single deployment with ECS info.

PUT /api/admin/deployments/[id] — Update deployment (super admin only).

DELETE /api/admin/deployments/[id] — Remove deployment (super admin only).

POST /api/admin/deployments/[id]/action — Execute lifecycle action (super admin only).

Body: `{ action: "deploy"|"start"|"stop"|"restart"|"rebuild"|"destroy" }`.

Returns: Generated Docker commands, SSH connection details, deployment config.

Admin ECS Endpoints

GET /api/admin/ecs — List ECS instances with deployment counts (super admin only).

POST /api/admin/ecs — Register ECS instance (super admin only).

GET /api/admin/ecs/[id] — ECS instance detail with all deployments.

PUT /api/admin/ecs/[id] — Update ECS instance (super admin only).

DELETE /api/admin/ecs/[id] — Remove ECS instance (only if zero deployments).

GET /api/admin/ecs/[id]/health — Health check (partial mock; SSH-based metrics pending).

GET /api/admin/ecs/credentials — List credential metadata (sanitized, no secrets exposed).

PUT /api/admin/ecs/credentials — Update SSH credentials (super admin only).

10. Security Considerations

Current Security Posture

Authentication: Cookie-based sessions with httpOnly flag prevent JavaScript access to session tokens. Secure flag enabled in production ensures HTTPS-only transmission. SameSite=lax prevents CSRF attacks.

Password Storage: Passwords are hashed using `crypto.scryptSync` with a 16-byte random salt and 64-byte derived key length. This is a NIST-recommended key derivation function resistant to brute-force attacks.

API Protection: All admin endpoints are wrapped with `withAuth()` middleware that validates sessions server-side. Super-admin-only endpoints return 403 for insufficient privileges. Public API endpoints (chat, obra) are intentionally open for workshop use.

Rate Limiting: Nginx enforces 10 req/s for API endpoints and 5 req/min for login attempts, per IP address. This provides basic protection against brute-force and denial-of-service attacks.

Container Hardening: Docker containers run as a non-root user (nextjs, UID 1001). The application port (3000) is bound to localhost only (`127.0.0.1:3000`), accessible only through the Nginx reverse proxy.

Input Validation: API routes validate required fields and return 400 errors for malformed requests. Document ID lookups validate against known patterns before accessing the filesystem.

Areas for Production Hardening

- **SSH key encryption:** ECS SSH keys are currently stored in the database. Production deployment should encrypt these at rest using a key management service.
- **Session rotation:** The current implementation does not rotate session tokens on privilege-sensitive operations. Consider implementing token rotation for critical actions.
- **Audit logging:** Admin actions (document uploads, deployment changes, credential updates) should be logged to a separate audit trail.
- **HTTPS enforcement:** The application should enforce HTTPS at the application level (not just Nginx redirect) for defense in depth.
- **OpenRouter API key management:** Per-LGU API keys are stored in the deployment record. Consider encrypting these and implementing key rotation.

11. Philippine Legal Domain Specificity

PILLAR is deeply specialized for Philippine local government law. This specialization is visible across multiple layers:

LGU Classification: The deployment system supports all four Philippine LGU types (municipality, city, component city, highly urbanized city) and all six income classes (1st through 6th), which determine legislative authority and resource allocation.

Legal Document Types: The knowledge base accommodates the five primary legal document types relevant to local governance: R.A. 7160 (Local Government Code), municipal ordinances, IRR (Implementing Rules and Regulations), DILG Legal Opinions, and SC Jurisprudence.

Bilingual Processing: The BM25 tokenizer includes 30+ Filipino/Tagalog stop words alongside English stop words. The YALA chatbot is configured to respond in both English and Filipino. FAQ data contains bilingual answers.

Roman Numeral Handling: The IRR of R.A. 7160 uses Roman numeral rule numbering (I through XXXIX). The citation validator includes a complete Roman-to-Arabic conversion map for accurate index lookups.

Citation Format Awareness: Different document types use different citation formats. R.A. 7160 uses "Section XXX", ordinances use "Municipal Ordinance No. XX, S. YYYY" or "MO No. XX, S. YYYY", and IRR uses "Rule XX". The regex patterns account for these variations.

Sangguniang Bayan Context: The system understands the specific legislative body structure — Vice Mayor as presiding officer, 8 regular SB members, 2 ex-officio members (ABC and SK federation presidents), and the SB Secretary.

12. Roadmap & Future Enhancements

Immediate Priorities

SSH-Based Deployment Execution: The deployment action endpoint currently generates Docker commands but does not execute them remotely. Implementing SSH connectivity to ECS instances would enable true one-click deployment management.

Real ECS Health Monitoring: The health check endpoint currently returns database-derived container counts but placeholder values for CPU, memory, disk, and Docker metrics. SSH-based health checks would provide real infrastructure visibility.

Runtime BM25 Re-indexing: While LightRAG ingestion is wired for runtime document additions, the BM25 search index is still pre-built at build time. A runtime BM25 re-index trigger (similar to the LightRAG reindex endpoint) would enable fully dynamic knowledge base updates without image rebuilds.

Medium-Term Enhancements

Per-LGU Branding: Each LGU deployment should support customizable branding — municipality name, logo, color scheme, SB member profiles, and local context. This would enable true white-label deployments from a single codebase.

Multi-Language Support: Extending YALA and ELLA to support additional Philippine languages (Cebuano, Ilocano, Hiligaynon) would broaden accessibility.

Workshop Facilitator Tools: Real-time facilitator dashboard showing live participant activity, question queues, and polling capabilities during Sangguniang Bayan workshops.

Document Versioning: Track changes to ordinances over time, enabling amendment detection and historical comparison of legal provisions.

Long-Term Vision

Cross-LGU Ordinance Harmonization: Using the aggregated knowledge base across all LGUs to identify conflicting ordinances, suggest harmonization opportunities, and surface best-practice ordinance templates.

DILG Compliance Automation: Automatically checking draft ordinances against the latest DILG opinions and flagging potential compliance issues before enactment.

Citizen Engagement Analytics: Aggregating YALA chatbot interactions to identify the most common citizen concerns, informing LGU policy priorities and service improvements.

Provincial Legislative Reporting: Automated generation of province-wide legislative reports summarizing ordinance activity, compliance trends, and capacity development across all municipalities.

Appendix A: File Structure

```

PILLAR/
  Dockerfile                # Multi-stage Docker build
  docker-compose.yml        # Service orchestration with production profile
  .env.example              # Environment variable template
  nginx/
    nginx.conf              # Main Nginx configuration
    conf.d/pillar.conf      # Subdomain routing for 4 domains
  src/
    app/
      page.tsx              # Landing page
      layout.tsx            # Root layout
      globals.css           # Global styles
      ella/page.tsx         # ELLA legal research UI
      obra/page.tsx         # OBRA ordinance drafting UI
      yala/page.tsx         # YALA citizen chatbot UI
      admin/
        layout.tsx          # Admin shell with auth check + nav
        page.tsx            # Admin dashboard (analytics)
        login/page.tsx      # Login form
        kb/page.tsx         # KB management
        kb/upload/page.tsx  # Document upload
        deployments/page.tsx # Deployment fleet view
        deployments/new/page.tsx # New deployment form
        ecs/page.tsx        # ECS infrastructure view
        ecs/new/page.tsx    # New ECS instance form
      api/
        chat/route.ts       # ELLA/YALA chat (SSE streaming)
        obra/draft/route.ts # Ordinance draft generation
        obra/review/route.ts # Compliance review
        obra/extract/route.ts # PDF/image text extraction
        documents/[id]/route.ts # Document full-text lookup
        health/route.ts     # Service health check
        auth/login/route.ts # Session creation
        auth/logout/route.ts # Session destruction
        auth/me/route.ts    # Current user lookup
        admin/kb/documents/route.ts # KB document CRUD
        admin/kb/upload/route.ts # File upload
        admin/kb/reindex/route.ts # LightRAG ingestion trigger
        admin/kb/stats/route.ts # KB aggregate statistics
        admin/deployments/route.ts # Deployment CRUD
        admin/deployments/[id]/action/route.ts # Docker lifecycle actions
        admin/ecs/route.ts   # ECS instance CRUD
        admin/ecs/[id]/health/route.ts # ECS health check
        admin/ecs/credentials/route.ts # SSH credential management
    lib/
      db.ts                 # SQLite setup, schema, password hashing
      auth.ts               # Session management
      auth-middleware.ts    # API route auth wrapper
      logger.ts             # Workshop interaction logging
      workshop-config.ts    # Module configuration
      topic-classifier.ts   # Message topic classification
      citation-doc-id.ts    # Citation-to-document ID mapping
      ai/
        rag.ts              # Hybrid RAG pipeline (LightRAG + BM25)
        lightrag.ts         # LightRAG HTTP client
        llm.ts              # OpenAI SDK (OpenRouter) configuration
        prompts.ts          # ELLA, OBRA, YALA system prompts
        citation-validator.ts # Post-generation citation verification
    data/
      search-index.json     # Pre-built BM25 index (~12 MB)
      yala-knowledge-base.json # Municipal services knowledge base

```

```
faq-data.json           # FAQ content (bilingual)
faq-questions.json      # FAQ question index
sanggungan-members.json # SB member profiles
documents/              # 2,634 individual document JSON files
types/
  index.ts              # All TypeScript interfaces and types
scripts/
  ingest.ts             # 6-step document ingestion pipeline
  build-index.ts         # BM25 index builder
parsers/
  ra7160-parser.ts      # R.A. 7160 section parser
  ordinance-parser.ts   # Ordinance file parser
  irr-parser.ts          # IRR rule parser
  legal-opinion-parser.ts # DILG opinion parser
  jurisprudence-parser.ts # SC jurisprudence parser
```

Appendix B: Default Configuration Values

Setting	Default	Notes
LLM Model	qwen/qwen3-235b-a22b	Qwen 3 235B MoE via OpenRouter
Chat Temperature	0.3	Low for factual grounding
Draft Temperature	0.5	Moderate for creative legal drafting
Review Temperature	0.1	Very low for deterministic compliance
Max Stream Tokens	2,048	Streaming responses
Max Completion Tokens	4,096	Non-streaming (draft/review)
BM25 K1	1.2	Term frequency saturation
BM25 B	0.75	Document length normalization
Context Budget	4,000 chars	Total RAG context for LLM
LightRAG top_k (ELLA/OBRA)	8	Knowledge graph results
LightRAG top_k (YALA)	5	Fewer results for citizen queries
BM25 Results (ELLA)	6	Full-text search results
BM25 Results (OBRA)	8	More results for drafting
BM25 Results (YALA)	3	Minimal for citizen chat
Section Number Boost	5x	Exact section match multiplier
Ordinance Number Boost	5x	Exact ordinance match multiplier
Topic Tag Boost	1.5x	Topic metadata match multiplier
Session Expiry	24 hours	Cookie-based session lifetime
Proxy Read Timeout	300s	Nginx timeout for SSE streams

Setting	Default	Notes
LightRAG Timeout	30s	HTTP client timeout
Health Check Timeout	5s	LightRAG health probe timeout
Rate Limit (API)	10 req/s	Per-IP rate limit
Rate Limit (Login)	5 req/min	Per-IP rate limit
Container Port	3000	Next.js listen port
Database Path	/data/pillar.db	SQLite in Docker volume

This document was generated from the PILLAR codebase (commit f18e60c) on June 8, 2026. For the latest version, refer to the repository documentation or regenerate from source.